

# 📦 Concept of Array

## 📖 Definition

An **array** is a collection of **multiple elements of the same data type**, stored in **continuous (contiguous) memory locations**, and referred to by a **single common name**.  
Each element inside the array is accessed using a number called an **index** (also called a **subscript**), which tells the computer **which position** in the array to look at.

## 💡 Why Do We Need Arrays?

Imagine you need to store the marks of 5 students. Without arrays, you'd have to create 5 separate variables:

```
int marks1 = 80, marks2 = 75, marks3 = 90, marks4 = 60, marks5 = 85;
```

This becomes **unmanageable** for 100 or 1000 students. An array solves this by letting you store **all of them under one name**:

```
int marks[5] = {80, 75, 90, 60, 85};
```

Now all 5 values live together, can be looped through easily, and are far simpler to manage.

## 🔑 Key Terminology

| Term              | Meaning                                                          |
|-------------------|------------------------------------------------------------------|
| Element           | A single value stored inside the array                           |
| Index / Subscript | The position number of an element (starts from <b>0</b> , not 1) |
| Size / Length     | The total number of elements the array can hold                  |
| Base Address      | The memory address of the <b>first element</b> (index 0)         |

## 📍 How Array Elements Sit in Memory

Array elements are stored **one right after another** in memory — this is what makes arrays fast to access. For `int marks[5] = {80, 75, 90, 60, 85};`

|      |      |      |      |      |
|------|------|------|------|------|
| 80   | 75   | 90   | 60   | 85   |
| [0]  | [1]  | [2]  | [3]  | [4]  |
| 1000 | 1002 | 1004 | 1006 | 1008 |

Notice the addresses increase by **2** here (assuming `int` takes 2 bytes) — each element sits **immediately after** the previous one, with **no gaps**. This is called **contiguous memory allocation**.

## 📌 Note

- All elements of an array must be of the **same data type** — you can't mix `int` and `float` in the same array.
- Array indexing **always starts at 0**, so an array of size `n` has valid indices from `0` to `n-1`.
- The array name itself represents the **base address** — this becomes very important later when working with pointers.
- Once declared, the **size of an array is fixed** — it cannot grow or shrink during the program.

## 💡 Fun Tip

Think of an array as a **row of numbered lockers** 🗄️ — same size, side-by-side, one name for the whole row!

# Array Declaration & Initialization

## Definition

**Declaring** an array means telling the compiler the array's **name**, **data type**, and **size**, so it can reserve the right amount of memory. **Initializing** means giving the array its **starting values**.

## Syntax — Declaration

```
data_type array_name[size];
```

| Part       | Meaning                                          | Example          |
|------------|--------------------------------------------------|------------------|
| data_type  | Type of value each element will hold             | int, float, char |
| array_name | The common name used to refer to the whole array | marks, arr       |
| size       | Total number of elements (must be a constant)    | 5, 100           |

### Examples

```
int marks[5];      // array of 5 integers
float price[10];   // array of 10 floats
char grade[3];     // array of 3 characters
```

## Syntax — Initialization

You can give values to an array **at the time of declaration** using curly braces { }:

```
data_type array_name[size] = {value0, value1, value2, ...};
```

| Way of Initializing    | Example                      | Explanation                                                           |
|------------------------|------------------------------|-----------------------------------------------------------------------|
| Full initialization    | int a[5] = {10,20,30,40,50}; | All 5 values given directly                                           |
| Size omitted           | int a[] = {10,20,30};        | Compiler automatically sets size to <b>3</b> (count of values given)  |
| Partial initialization | int a[5] = {10,20};          | First 2 elements = 10, 20 — <b>remaining 3 elements auto-set to 0</b> |
| All zero               | int a[5] = {0};              | All 5 elements become <b>0</b>                                        |

## Accessing & Modifying Array Elements

Once declared, individual elements are accessed using the **array name with an index in square brackets**: array\_name[index]

### Example Program

```
#include <stdio.h>

int main() {
    int marks[5] = {80, 75, 90, 60, 85};

    printf("First element : %d\n", marks[0]);
    printf("Third element : %d\n", marks[2]);

    marks[1] = 95; // modify the 2nd element
    printf("Updated 2nd element : %d\n", marks[1]);

    // Printing all elements using a loop
    for (int i = 0; i < 5; i++) {
        printf("marks[%d] = %d\n", i, marks[i]);
    }

    return 0;
}
```

### Output

```
First element : 80
Third element : 90
Updated 2nd element : 95
marks[0] = 80
marks[1] = 95
marks[2] = 90
marks[3] = 60
marks[4] = 85
```

## Note

- The **size** used in declaration must be a **constant** — you cannot use a variable whose value is decided at runtime (in standard C).
- Loops (especially for) are the **most common way** to access or fill every element of an array, since indices follow a simple pattern.
- Accessing an index **outside the valid range** (like marks[5] for a 5-element array) does **not give an error** in C — it silently reads/writes **garbage or unrelated memory**. This is a common and dangerous bug (see Limitations).

### \* Fun Tip

Index always starts from **0** — the "5th student" is actually at marks[4]!

# ➔ One-Dimensional (1-D) Array

## 📖 Definition

A **one-dimensional array** is the **simplest form** of an array — it stores elements in a **single row (single list)**, and each element is accessed using **just one index**. This is the type of array we have been using so far.

## 🔧 Syntax

```
data_type array_name[size];
```

Think of a 1-D array as a **single line of boxes**:

|     |     |     |     |
|-----|-----|-----|-----|
| 10  | 20  | 30  | 40  |
| [0] | [1] | [2] | [3] |

### 🔗 Example Program — Sum of Array Elements

```
#include <stdio.h>

int main() {
    int num[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum = sum + num[i];
    }

    printf("Sum of array elements = %d\n", sum);

    return 0;
}
```

### ▶ Output

```
Sum of array elements = 150
```

## 🔗 Common 1-D Array Operations

| Operation                                 | How                                                                           |
|-------------------------------------------|-------------------------------------------------------------------------------|
| <b>Traversal</b> (visiting every element) | <code>for(i=0; i&lt;n; i++) printf("%d", arr[i]);</code>                      |
| <b>Finding the largest element</b>        | Loop through, keep updating a "max" variable when a bigger value is found     |
| <b>Searching for a value</b>              | Loop through and compare each element with the target value                   |
| <b>Reversing</b>                          | Swap element at index <code>i</code> with element at index <code>n-1-i</code> |

### 🔗 Example: Finding the Largest Element

```
int arr[5] = {12, 45, 7, 89, 23};
int max = arr[0]; // assume first element is the largest, to start

for (int i = 1; i < 5; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}

printf("Largest element = %d\n", max);
```

### ▶ Output

```
Largest element = 89
```

## 📌 Note

- 1-D arrays are used when data has a **single, simple sequence** — a list of marks, temperatures, prices, etc.
- The total memory used by a 1-D array = `size × size_of(data_type)`. Example: `int arr[5]` on a system where `int` is 4 bytes → uses **20 bytes**.
- Passing a 1-D array to a function automatically passes its **base address** — the function works on the **original array**, not a copy.

### 💡 Fun Tip

1-D array = a single **train** ~~series~~ of boxes, one after another!

# Multi-Dimensional Array (2-D Array)

## Definition

A **multi-dimensional array** is an array of arrays — it stores data in the form of **rows and columns** (like a table or grid), instead of a single line. The most common form is the **two-dimensional (2-D) array**, which needs **two indices** to access an element: one for the **row**, one for the **column**.

## Syntax

```
data_type array_name[rows][columns];
```

### Example — Declaration & Initialization

```
int matrix[2][3] = {
    {1, 2, 3}, // row 0
    {4, 5, 6}  // row 1
};
```

## Visualizing a 2-D Array (2 rows × 3 columns)

|             |             |             |
|-------------|-------------|-------------|
| 1<br>[0][0] | 2<br>[0][1] | 3<br>[0][2] |
| 4<br>[1][0] | 5<br>[1][1] | 6<br>[1][2] |

The first index [row] picks the **row**, and the second index [column] picks the **column** inside that row. So matrix[1][2] means "row 1, column 2" → value **6**.

## How It's Actually Stored in Memory

Even though we *visualize* a 2-D array as a grid, computer memory is **always linear (one straight line)**. C stores 2-D arrays in **Row-Major Order** — meaning it stores the **entire first row**, then the **entire second row**, and so on, one after another.

|             |             |             |             |             |             |
|-------------|-------------|-------------|-------------|-------------|-------------|
| 1<br>[0][0] | 2<br>[0][1] | 3<br>[0][2] | 4<br>[1][0] | 5<br>[1][1] | 6<br>[1][2] |
|-------------|-------------|-------------|-------------|-------------|-------------|

### Example Program — Reading & Printing a 2-D Array

```
#include <stdio.h>

int main() {
    int matrix[2][3] = {{1,2,3}, {4,5,6}};

    // Two nested loops are needed: outer for rows, inner for columns
    for (int i = 0; i < 2; i++) { // rows
        for (int j = 0; j < 3; j++) { // columns
            printf("%d ", matrix[i][j]);
        }
        printf("\n"); // new line after each row
    }

    return 0;
}
```

### Output

```
1 2 3
4 5 6
```

## Note

- A 2-D array is really an **"array of rows"** — each row is itself a 1-D array.
- Accessing a 2-D array always needs **two nested loops** — the outer loop for rows, the inner loop for columns.
- C also supports arrays with **more than 2 dimensions** (e.g. int a[2][3][4]; is a 3-D array), though these are used far less often and follow the same row-major idea.
- A common use of 2-D arrays is representing a **matrix** in mathematics, or a **grid/table** of data (like a tic-tac-toe board or a seating chart).

### Fun Tip

2-D array = a **chessboard** ♠ – row number + column number finds any square!

# Character Arrays

## Definition

A **character array** is an array whose data type is `char` — meaning each element holds a **single character**. When a character array is used to store a sequence of characters that forms **text**, C treats it specially and calls it a **string** (covered fully on the next page).

## Syntax

```
char array_name[size];
```

### Example — Declaring & Filling a Character Array

```
char grade[3];

grade[0] = 'A';
grade[1] = '+';
grade[2] = '\0'; // null character marks the end
```

## How It Sits in Memory

|     |     |      |
|-----|-----|------|
| 'A' | '+' | '\0' |
| [0] | [1] | [2]  |

Each character occupies **1 byte**. The special character `'\0'` (called the **null character**, with ASCII value **0**) is used to mark **"this is where the text ends."** This becomes essential when we treat the array as a **string**.

## Two Ways to Initialize a Character Array

| Method                 | Example                                                 | Note                                                             |
|------------------------|---------------------------------------------------------|------------------------------------------------------------------|
| Character by character | <code>char name[6] = {'A','m','a','n',' ','\0'};</code> | You must <b>manually add</b> <code>'\0'</code> at the end        |
| As a string literal    | <code>char name[6] = "Aman ";</code>                    | C <b>automatically adds</b> <code>'\0'</code> at the end for you |

### Example Program

```
#include <stdio.h>

int main() {
    char letters[5] = {'H', 'e', 'l', 'l', 'o'};

    // Printing each character using its index
    for (int i = 0; i < 5; i++) {
        printf("%c", letters[i]);
    }
    printf("\n");

    return 0;
}
```

### Output

```
Hello
```

Notice this array has **no '\0'** — it was printed only because we controlled the loop with a fixed count (`i < 5`). Without a null character, C has **no way of knowing** where the text should stop if we try to print it as a string (using `%s`) — this is exactly why strings need `'\0'`.

## Note

- A character array is just a **normal array** with `char` as its type — it follows all the same rules as other arrays.
- It becomes a **"string"** only when it is **null-terminated** (ends with `'\0'`) and used with string-handling functions or `%s`.
- Always leave **one extra space** in the size for the null character — `char name[6]` can hold a 5-letter word **plus** the `'\0'`.

### Fun Tip

`'\0'` = the **"THE END"** 🎬 sign for text in C!

# Strings in C

## Definition

A **string** in C is simply a **character array that ends with the null character '\0'**. C has **no separate "string" data type** like some other languages — it uses char arrays to represent text.

## Declaring & Initializing Strings

```
char city[6] = "Delhi";    // 5 letters + 1 auto '\0' = needs size 6
char city2[] = "Delhi";    // size auto-decided as 6
```

Reading a string from the user uses %s with scanf, or the safer fgets function:

```
char name[20];
scanf("%s", name);    // NOTE: no '&' needed, array name is already an address
```

## Common String Handling Functions (string.h)

| Function          | Purpose                                               | Example                   | Result         |
|-------------------|-------------------------------------------------------|---------------------------|----------------|
| strlen(s)         | Finds the <b>length</b> of a string (excluding '\0' ) | strlen("Hello")           | 5              |
| strcpy(dest, src) | <b>Copies</b> one string into another                 | strcpy(a, "Hi")           | a becomes "Hi" |
| strcat(s1, s2)    | <b>Joins (concatenates)</b> s2 onto the end of s1     | strcat("Good", "Morning") | "GoodMorning"  |
| strcmp(s1, s2)    | <b>Compares</b> two strings                           | strcmp("cat", "cat")      | 0 (equal)      |

### Example Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char first[20] = "Good";
    char second[20] = "Morning";

    printf("Length of first : %d\n", strlen(first));

    strcat(first, second);    // joins second onto first
    printf("After strcat : %s\n", first);

    if (strcmp(first, "GoodMorning") == 0) {
        printf("Strings match!\n");
    }

    return 0;
}
```

### Output

```
Length of first : 4
After strcat : GoodMorning
Strings match!
```

## String vs Plain Character Array

| Aspect                      | Character Array         | String                   |
|-----------------------------|-------------------------|--------------------------|
| Ends with '\0' ?            | Not necessarily         | <b>Always must</b>       |
| Can use %s, strlen(), etc.? | Only if null-terminated | <b>Yes</b>               |
| Example                     | {'H', 'i'} (no '\0' )   | "Hi" (has '\0' built in) |

## Note

- Every string literal written in " " (double quotes) **automatically gets a '\0'** added by the compiler — you never add it yourself for a literal.
- %c is used to print a **single character**; %s is used to print an **entire string** (it keeps printing until it hits '\0').
- scanf("%s", name) stops reading at the **first space** — it cannot read a full sentence with spaces. Use fgets() for that.
- Always remember to #include <string.h> before using functions like strlen, strcpy, strcat, strcmp.

### Fun Tip

A string is just a **char array with a "The End" sticker '\0'** at the end! 🗑

# ⚠ Limitations of Array

## 📖 Definition

While arrays are extremely useful, they have several **built-in limitations** in C. Knowing these is important because they explain **why pointers, dynamic memory allocation, and other data structures** are needed later in programming.

## 🔗 Main Limitations

| # | Limitation                    | Explanation                                                                                                                                                                                                     |
|---|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Fixed Size                    | The size of an array must be decided <b>at the time of declaration</b> and <b>cannot be changed</b> later — it can't grow if you need more space, and wastes memory if you need less.                           |
| 2 | Same Data Type Only           | All elements <b>must be of one data type</b> . You cannot store a mix of <code>int</code> , <code>float</code> , and <code>char</code> in the same array.                                                       |
| 3 | No Automatic Bounds Checking  | C does <b>not check</b> if an index is within valid range. Accessing <code>arr[10]</code> in a 5-element array doesn't stop the program — it reads/writes <b>unrelated memory</b> , causing unpredictable bugs. |
| 4 | Memory Wastage                | If you declare <code>int arr[100]</code> but only use 10 elements, the memory for the other 90 is <b>reserved but wasted</b> .                                                                                  |
| 5 | Costly Insertion / Deletion   | To insert or delete an element in the <b>middle</b> , all the following elements must be <b>manually shifted</b> — this takes extra time and code.                                                              |
| 6 | Contiguous Memory Requirement | An array needs one single <b>continuous block</b> of memory. For very large arrays, the system may not find enough contiguous free memory, even if enough <i>total</i> free memory exists elsewhere.            |

### 🔍 Example: Out-of-Bounds Access (Dangerous!)

```
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};

    printf("%d\n", arr[5]); // index 5 does NOT exist (valid: 0,1,2)
    // No compiler error! Prints garbage / unpredictable value
    // This is called "Undefined Behaviour"

    return 0;
}
```

## 📌 Note — Why This Matters

- These limitations are exactly why C later introduces **pointers** and **dynamic memory allocation** (`malloc`, `calloc`) — to create arrays whose size can be decided and changed **while the program is running**.
- Other programming languages solve some of these problems using **dynamic arrays / lists** (like Python lists or Java ArrayLists), which grow automatically — but internally, they still rely on the same array concept.
- As a beginner, the most important habit to build is: **always double-check your loop conditions** (`i < n`, not `i <= n`) to avoid going out of bounds.

## 🔄 Quick Recap of the Whole Array Topic

| Topic            | One-Line Summary                                                                  |
|------------------|-----------------------------------------------------------------------------------|
| Concept of Array | Same-type elements, stored together, one name, index-based access                 |
| Declaration      | <code>type name[size];</code> — reserves memory in advance                        |
| 1-D Array        | Single row of elements, one index                                                 |
| 2-D Array        | Rows & columns, two indices, stored row-by-row in memory                          |
| Character Array  | Array of <code>char</code> ; becomes a "string" with <code>'\0'</code> at the end |
| Strings          | Null-terminated char arrays; handled with <code>string.h</code> functions         |
| Limitations      | Fixed size, single type, no bounds checking, costly insert/delete                 |

### 💡 Fun Tip

An array is a great **starting tool** — but not always the **final** tool. That's where pointers come in next! 🧠